CSA4704 – DEEP LEARNING
CO1 ASSIGNMENT 1
AgriVision Crop-Disease Screening System
J.SUBHAASHREE (192424008)

Dataset Summary

Property	Observed value
Image tensor shape	(8700, 64, 64)
Flattened feature matrix	(8700, 4096)
Pixel range after scaling	0.0 to 1.0
Usable images	8,700
Excluded artifacts	7 checkpoint/artifact records
Number of classes	29
Class balance	300 images per class
Train / validation / test	6,090 / 1,305 / 1,305

Task 1 – Problem Structure and Data Representation

1. Theory explanation

AgriVision is a supervised, multi-class image-classification problem. Each input is a 64 × 64 grayscale leaf image and the target is one crop–disease label. A model must learn a mapping from image pixels to labels while generalising to photographs it has not seen. The representation determines which patterns are visible to the algorithm: flattening makes standard machine-learning models easier to use, but removes the explicit two-dimensional neighbourhood structure used by convolutional networks.

2. Mathematical formulas

- Image tensor: X ∈ R^(N×64×64); flattened matrix: X_flat ∈ R^(N×4096).
- Mini-batch of 32 flattened images: X_batch ∈ R^(32×4096).
- L^1 norm: $||x||_1 = \Sigma_i |x_i|$; L^2 norm: $||x||_2 = \sqrt{(\Sigma_i x_i^2)}$; $L\infty$ norm: $||x||\infty = \max_i |x_i|$.
- Min–max pixel scaling used here: x_scaled = x / 255.

3. Python code used

```
# Demonstrate shapes and vector norms on one image
one=X[0]
L1=float(np.linalg.norm(one, ord=1)); L2=float(np.linalg.norm(one, ord=2)); Linf=float(np.linalg.norm(one, ord=np.inf))
print(f'Dataset X: {X.shape}; one image: {one.shape}; batch of 32: {X[:32].shape}; labels: {y.shape}')
print(f'Norms for the same image vector: L1={L1:.6f}, L2={L2:.6f}, L-infinity={Linf:.6f}')
print('L1 measures total absolute intensity; L2 measures overall magnitude; L-infinity captures the largest pixel value.')
fig,ax=plt.subplots(1,2,figsize=(8,3)); ax[0].imshow(X_img[0],cmap='gray'); ax[0].set_title(class_names[y[0]]);
ax[0].axis('off'); ax[1].hist(X[0],bins=30); ax[1].set_title('Pixel values after /255'); plt.tight_layout(); plt.show()
```

4. Calculated results and output

1. Image Tensor and Flattened Matrix

$$X \in \mathbb{R}^{N \times 64 \times 64}$$

$$N = 8700$$

$$64 \times 64 = 4096$$

$$X_{\text{flat}} \in \mathbb{R}^{N \times 4096}$$

$$\boxed{X_{\text{flat}} \in \mathbb{R}^{8700 \times 4096}}$$

2. Mini-Batch of 32 Images

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 4096}$$

$$32 \times 4096 = 131072$$

$$\boxed{X_{\text{batch}} \in \mathbb{R}^{32 \times 4096}}$$

3. L1 Norm

$$||x||_1 = \sum_{i=1}^{n} |x_i|$$

$$||x||_1 = 2394.694092$$

$$\boxed{||x||_1 = 2394.6941}$$

4. L2 Norm

$$||x||_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

$$||x||_2 = 38.909485$$

$$\boxed{||x||_2 = 38.9095}$$

5. L-Infinity Norm

$$||x||_\infty = \max_i |x_i|$$

$$||x||_\infty = 0.807843$$

$$\boxed{||x||_\infty = 0.8078}$$

6. Min-Max Pixel Scaling

$$x_{\text{scaled}} = \frac{x}{255}$$

For $x = 128$:

$$x_{\text{scaled}} = \frac{128}{255}$$

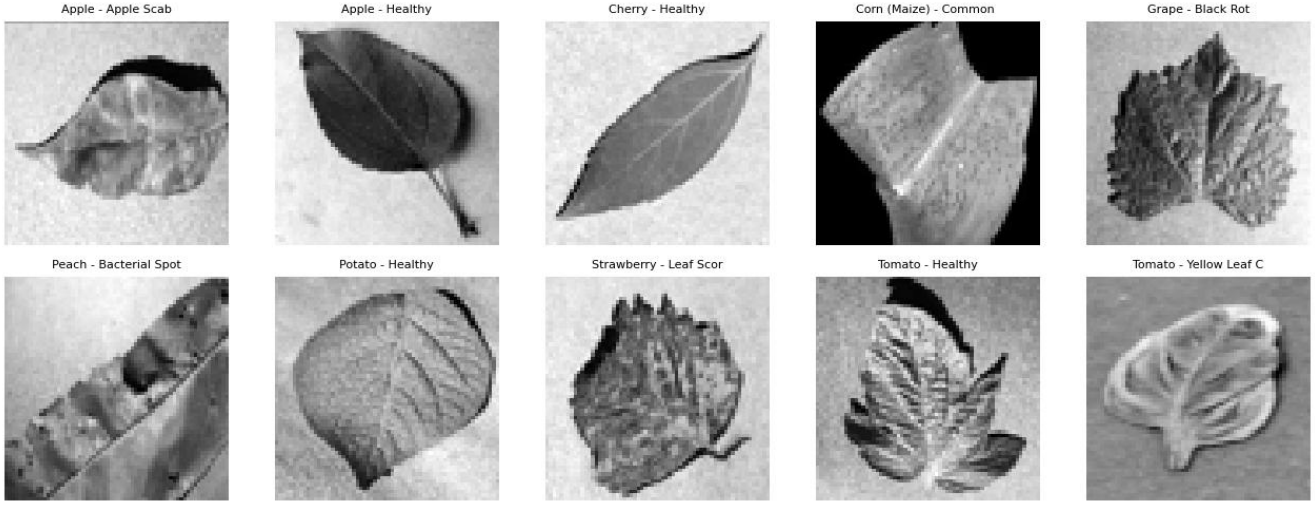
$$x_{\text{scaled}} = 0.50196078$$

$$\boxed{x_{\text{scaled}} = 0.5020}$$

5. What the result means for AgriVision

The observed dataset has 8,700 usable images, 4,096 pixel features per image, and 29 balanced classes (300 images per class). Resizing reduces computational cost but may discard small disease spots. Normalisation makes pixel magnitudes comparable. The train/validation/test split prevents evaluation on training examples and helps avoid optimistic performance estimates.

6. Plot(s)



Dataset as grayscale image tensor: (8700, 64, 64) (N, 64, 64)

One image: (64, 64) matrix; flattened feature vector: (4096,)

Mini-batch of 32: (32, 4096) (32, 4096)

Integer labels for mini-batch: (32,) (32,)

One label is a scalar class index; one-hot labels would have shape: (29,)

For CNNs, a channel dimension can be added: (8700, 64, 64, 1)

First image feature-vector norms: L1=2394.6941, L2=38.9095, L-infinity=0.8078

Pixel range before scaling: 0 to 255

Pixel range after scaling: 0.0 to 1.0

Flattening changes shape only (4096 features); it removes explicit 2-D neighbourhood structure.

Task 2 – Feature-Space Structure Using PCA/SVD

1. Theory explanation

Principal Component Analysis (PCA) rotates correlated pixel features into orthogonal directions ranked by variance. The first components preserve the largest possible variance, which can reduce dimensionality and memory. High correlation among nearby pixels often means the original 4,096 features contain redundancy. However, variance is not the same as disease-discriminative information, so PCA retention must be compared with classification metrics.

2. Mathematical formulas

- Center the training matrix: $X_c = X - \mu$, where μ is the training-feature mean.
- Covariance: $C = X_c^T X_c/(n-1)$.
- SVD: $X_c = U\Sigma V^T$; covariance eigenvalues satisfy $\lambda_i = s_i^2/(n-1)$.
- Explained variance ratio: $r_i = \lambda_i / \Sigma_j \lambda_j$; cumulative ratio $R_k = \Sigma_{i=1}^{k} r_i$.
- Reconstruction: $\hat{X}_k = X_c\, V_k\, V_k^T + \mu$; reconstruction MSE = mean$((X - \hat{X}_k)^2)$.

3. Python code used

```
# Small hand-worked 2x2 example: A=[[2,0],[0,1]]; A^T A=diag(4,1)
A_small=np.array([[2.,0.],[0.,1.]])
AtA=A_small.T@A_small
eigvals,eigvecs=np.linalg.eigh(AtA)
svals=np.sqrt(np.maximum(eigvals,0))[::-1]
print('A^T A =\n',AtA); print('Eigenvalues (ascending):',eigvals); print('Singular values (descending):',svals)
print('For this example, rank-1 keeps energy 4/(4+1)=80%.')
Xtr, Xtemp, ytr, ytemp=train_test_split(X,y,test_size=0.30,random_state=SEED,stratify=y)
Xval, Xte, yval, yte=train_test_split(Xtemp,ytemp,test_size=0.50,random_state=SEED,stratify=ytemp)
# PCA fitted to training data, then transform all partitions
pca=PCA(n_components= min(100, Xtr.shape[0]-1, Xtr.shape[1]), svd_solver='randomized', random_state=SEED)
t0=time.perf_counter(); pca.fit(Xtr); pca_fit_time=time.perf_counter()-t0
Ztr=pca.transform(Xtr); Zval=pca.transform(Xval); Zte=pca.transform(Xte)
varcum=np.cumsum(pca.explained_variance_ratio_)
print(f'Train/validation/test: {Xtr.shape[0]}/{Xval.shape[0]}/{Xte.shape[0]}')
for k in [10,25,50,100]:
 if k<=len(varcum):
  print(f'k={k:3d}: explained variance={varcum[k-1]:.4f}, reconstruction MSE={np.mean((Xte-
pca.inverse_transform(np.pad(Zte[:,:k],((0,0),(0,Zte.shape[1]-k)))))**2):.6f}')
print(f'PCA fit time: {pca_fit_time:.2f} seconds')
plt.figure(figsize=(7,4)); plt.plot(np.arange(1,len(varcum)+1),varcum); plt.xlabel('Number of principal components');
plt.ylabel('Cumulative explained variance'); plt.title('AgriVision PCA explained variance'); plt.grid(True);
plt.tight_layout(); plt.show()
# Visualise first 8 principal components
fig,axs=plt.subplots(2,4,figsize=(9,5))
for i,a in enumerate(axs.flat): a.imshow(pca.components_[i].reshape(64,64),cmap='coolwarm'); a.set_title(f'PC {i+1}');
a.axis('off')
plt.suptitle('Leading principal components (pixel-weight patterns)'); plt.tight_layout(); plt.show()
```

4. Calculated results and output

1. Center the Training Matrix

$$X_c = X - \mu$$

2. Covariance Matrix

$$C = \frac{X_c^T X_c}{n-1}$$

3. SVD and Eigenvalues

$$X_c = U \Sigma V^T$$

$$\lambda_i = \frac{s_i^2}{n-1}$$

Small example:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\boxed{\lambda_1 = 4, \quad \lambda_2 = 1}$$

$$s_1 = \sqrt{4} = 2, \qquad s_2 = \sqrt{1} = 1$$

4. Explained Variance Ratio

$$r_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

For $k = 1$:

$$r_1 = \frac{4}{4+1} = \frac{4}{5}$$

$$\boxed{r_1 = 0.80 = 80\%}$$

5. Cumulative Explained Variance

$$R_k = \sum_{i=1}^{k} r_i$$

For $k = 1$:

$$R_1 = 0.80$$

$$\boxed{R_1 = 80\%}$$

6. Reconstruction

$$\hat{X}_k = X_c V_k V_k^T + \mu$$

7. Reconstruction Mean Squared Error

$$\text{MSE} = \text{mean}\left((X - \hat{X}_k)^2\right)$$

5. What the result means for AgriVision

In the small hand-worked example A = [[2,0],[0,1]], A^TA = diag(4,1), so singular values are 2 and 1; rank-1 energy retained is 4/(4+1) = 80%. For AgriVision, the executed notebook reports cumulative explained variance of 68.14% at k=10, 77.45% at k=25, 83.18% at k=50, and 88.03% at k=100. At k=100, reconstruction MSE is about 0.004195.

6. Plot(s)

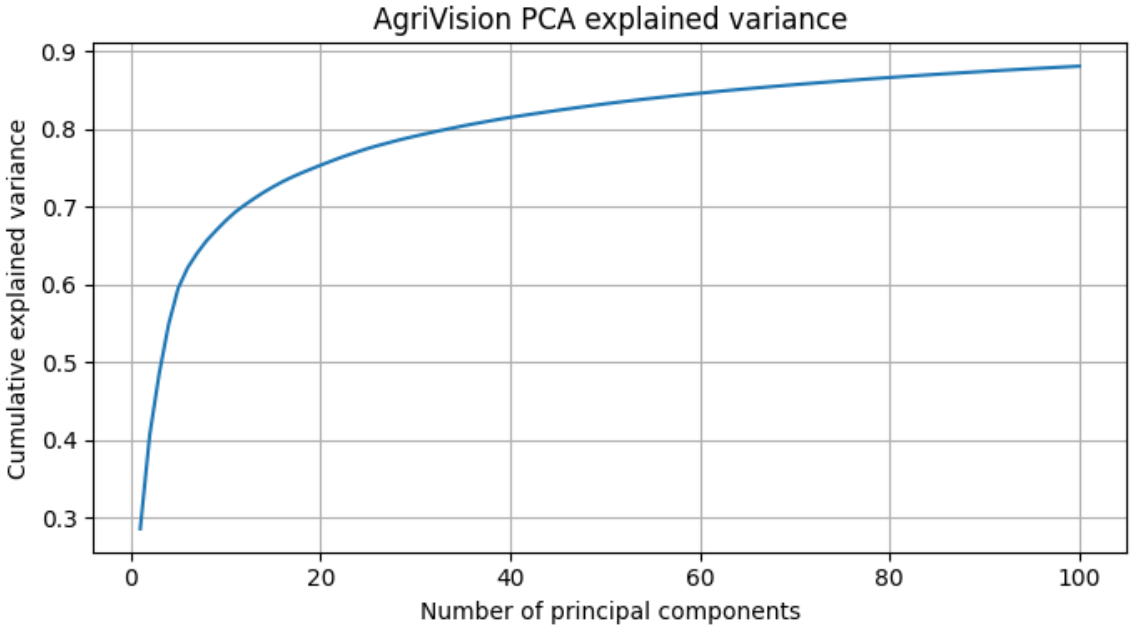


Figure: PCA Explained Variance

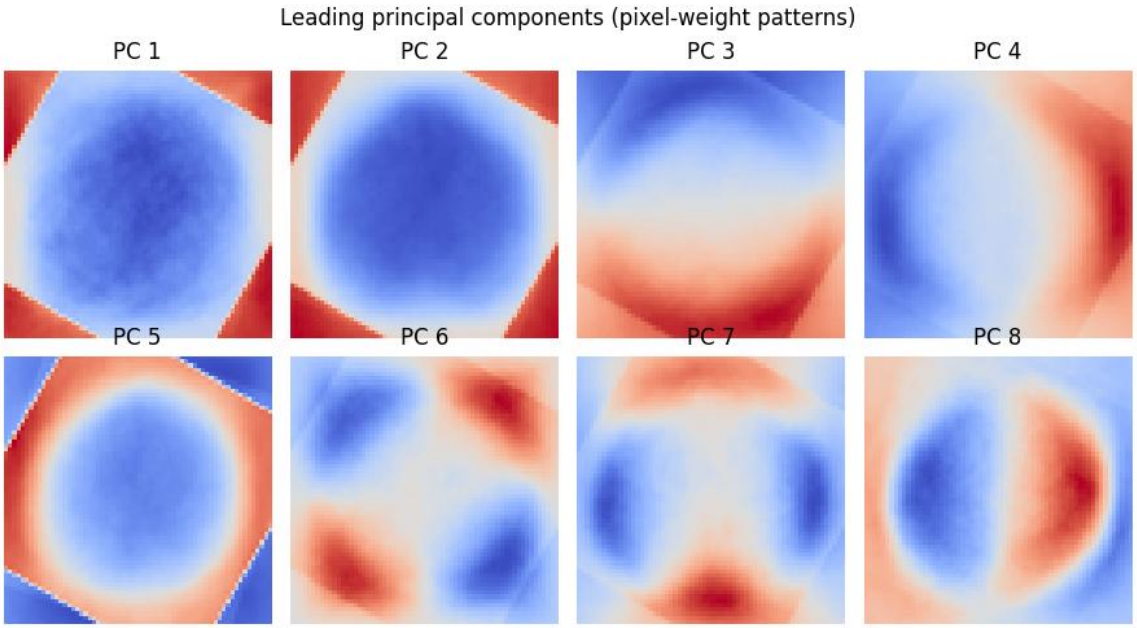


Figure: Principal Components

Task 3 – Probability and Information Theory

1. Theory explanation

Entropy measures uncertainty in the true class distribution. Cross-entropy measures the penalty assigned to a model's predicted probabilities when evaluated against true labels. KL divergence measures how different a predicted distribution q is from a reference distribution p. Cross-entropy and KL divergence are related: for fixed true distribution p, minimising cross-entropy over q is equivalent to minimising KL(p||q), because H(p) is constant.

2. Mathematical formulas

- Entropy: $H(p) = -\Sigma_i\ p_i\ \log_2(p_i)$.
- Cross-entropy: $H(p,q) = -\Sigma_i\ p_i\ \log_2(q_i)$.
- KL divergence: $D_{KL}(p||q) = \Sigma_i\ p_i\ \log_2(p_i/q_i) = H(p,q) - H(p)$.
- Maximum entropy for K equally likely classes: $H_{max} = \log_2(K)$.

3. Python code used

```
# Entropy / KL illustrative numeric example (not a model score)
def entropy_bits(p):
 p=np.asarray(p,dtype=float); p=p[p>0]; return float(-np.sum(p*np.log2(p)))
def cross_entropy_bits(p,q):
 p=np.asarray(p,dtype=float); q=np.asarray(q,dtype=float); return float(-np.sum(p*np.log2(np.clip(q,1e-15,1))))
def kl_bits(p,q): return cross_entropy_bits(p,q)-entropy_bits(p)
p_demo=np.array([0.5,0.3,0.2]); q_demo=np.array([0.7,0.2,0.1])
print('H(p)=',round(entropy_bits(p_demo),6),'bits')
print('H(p,q)=',round(cross_entropy_bits(p_demo,q_demo),6),'bits')
print('KL(p||q)=',round(kl_bits(p_demo,q_demo),6),'bits')
print('KL(q||p)=',round(kl_bits(q_demo,p_demo),6),'bits (different direction, generally unequal)')
print('Balanced 29-class entropy upper bound log2(29)=',round(math.log2(C),6),'bits')
```

4. Calculated results and output

1. Entropy

$$H(p) = -\sum_i p_i \log_2(p_i)$$

$$\boxed{H(p) = 1.485475 \text{ bits}}$$

2. Cross-Entropy

$$H(p,q) = -\sum_i p_i \log_2(q_i)$$

$$\boxed{H(p,q) = 1.618251 \text{ bits}}$$

3. KL Divergence

$$D_{KL}(p\|q) = \sum_i p_i \log_2\left(\frac{p_i}{q_i}\right)$$

$$D_{KL}(p\|q) = H(p,q) - H(p)$$

$$= 1.618251 - 1.485475$$

$$\boxed{D_{KL}(p\|q) = 0.132776 \text{ bits}}$$

4. Reverse KL Divergence

$$D_{KL}(q\|p) = \sum_i q_i \log_2\left(\frac{q_i}{p_i}\right)$$

$$\boxed{D_{KL}(q\|p) = 0.122806 \text{ bits}}$$

5. Maximum Entropy for 29 Classes

$$H_{\max} = \log_2(K)$$

Substitute $K = 29$:

$$H_{\max} = \log_2(29)$$

$$\boxed{H_{\max} = 4.857981 \text{ bits}}$$

5. What the result means for AgriVision

It gives H(p)=1.485475 bits, H(p,q)=1.618251 bits, and KL(p||q)=0.132775 bits. Reversing the direction gives KL(q||p)=0.122806 bits, illustrating asymmetry. For 29 equally likely classes, maximum entropy is $\log_2(29)$=4.857981 bits.

Task 4 – Numerical Computation and Conditioning

1. Theory explanation

Softmax converts logits into probabilities. Directly exponentiating very large logits can overflow, while very negative values can underflow. Subtracting the largest logit before exponentiation changes every exponential by the same positive factor, so the final probability ratios stay the same while the computation becomes stable. Poorly conditioned covariance matrices can also make optimisation sensitive to learning rates and slow convergence.

2. Mathematical formulas

- Softmax: $p_i = \exp(z_i)/\Sigma_j \exp(z_j)$.
- Stable softmax: $p_i = \exp(z_i - m)/\Sigma_j \exp(z_j - m)$, where $m = \max_j z_j$.
- Log-sum-exp: $\log(\Sigma_i \exp(z_i)) = m + \log(\Sigma_i \exp(z_i - m))$.
- Condition number (2-norm for positive-definite covariance): $\kappa(C) = \lambda_max(C)/\lambda_min(C)$.

3. Python code used

```
z=np.array([1000.,999.,995.])
with np.errstate(over='ignore', invalid='ignore', divide='ignore'):
 naive=np.exp(z)/np.sum(np.exp(z))
shift=z-np.max(z); stable=np.exp(shift)/np.sum(np.exp(shift))
logsumexp=np.max(z)+np.log(np.sum(np.exp(z-np.max(z))))
print('Naive softmax:',naive)
print('Stable softmax:',stable)
print('Log-sum-exp:',logsumexp,'; stable probabilities sum=',stable.sum())
# covariance condition estimate using PCA spectrum; use positive eigenvalues only
vals=pca.explained_variance_
pos=vals[vals>1e-12]
print('Approx. covariance condition number over retained PCA directions:',float(pos.max()/pos.min()))
```

4. Calculated results and output

1. Softmax

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

For $z = [1000, 999, 995]$:

$$p_i = \frac{e^{z_i}}{e^{1000} + e^{999} + e^{995}}$$

$$\boxed{p = [\text{NaN}, \text{NaN}, \text{NaN}]}$$

2. Stable Softmax

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}$$

$$m = \max(z) = 1000$$

$$z - m = [0, -1, -5]$$

$$p_1 = \frac{e^0}{e^0 + e^{-1} + e^{-5}} = \frac{1}{1 + 0.367879 + 0.006738}$$

$$p_1 = \frac{1}{1.374617} = 0.727475$$

$$p_2 = \frac{e^{-1}}{1 + e^{-1} + e^{-5}} = \frac{0.367879}{1.374617} = 0.267623$$

$$p_3 = \frac{e^{-5}}{1 + e^{-1} + e^{-5}} = \frac{0.006738}{1.374617} = 0.004902$$

$$\boxed{p = [0.727475,\ 0.267623,\ 0.004902]}$$

3. Log-Sum-Exp

$$\log\left(\sum_i e^{z_i}\right) = m + \log\left(\sum_i e^{z_i - m}\right)$$

$$= 1000 + \log(e^0 + e^{-1} + e^{-5})$$

$$= 1000 + \log(1 + 0.367879 + 0.006738)$$

$$= 1000 + \log(1.374617)$$

$$\boxed{\text{Log-Sum-Exp} = 1000.318175}$$

4. Covariance Condition Number

$$\kappa(C) = \frac{\lambda_{\max}(C)}{\lambda_{\min}(C)}$$

Using the retained PCA directions:

$$\boxed{\kappa(C) \approx 455.138489}$$

5. What the result means for AgriVision

For z=[1000,999,995], naive softmax returns NaN values due to overflow, whereas stable softmax returns approximately [0.727475, 0.267623, 0.004902], summing to 1. The reported condition number over retained PCA directions is approximately 455.14. This suggests noticeable scale differences among directions; standardisation, PCA, and conservative learning rates can improve optimisation stability.

Task 5 – Compare Three Baseline Learning Strategies

1. Theory explanation

The three baselines represent different inductive biases. Logistic regression learns a linear decision boundary and is relatively compact. k-nearest neighbours classifies using local similarity and has almost no fitting phase, but prediction cost grows with stored examples. A decision tree learns nonlinear rules and can capture feature interactions, but deep trees can overfit. PCA changes the geometry of the data and may remove noise or useful discriminative details.

2. Mathematical formulas

- Multiclass logistic regression: $P(y=c|x)=\exp(w_c^T x+b_c)/\Sigma_j \exp(w_j^T x+b_j)$.
- k-NN: $\hat{y}$ = majority label among the k nearest training samples under a chosen distance metric.
- Decision tree split impurity commonly uses Gini: $G = 1 - \Sigma_c\ p_c^2$.
- Accuracy = correct predictions / all predictions; macro-F1 = unweighted mean of per-class F1.

3. Python code used

```
def make_model(name):
 if name=='Logistic Regression': return SGDClassifier(loss='log_loss', max_iter=20, tol=None, alpha=1e-4,
random_state=SEED)
 if name=='k-NN': return KNeighborsClassifier(n_neighbors=5, weights='distance', n_jobs=1)
 if name=='Random Forest': return RandomForestClassifier(n_estimators=5, max_depth=10, min_samples_leaf=1,
max_features='sqrt', n_jobs=1, random_state=SEED)
 raise ValueError(name)
# Dataset-scale experiments. PCA is precomputed on training data; raw subset uses 256 evenly spaced pixels; PCA is fit
from all 4096 pixel inputs.
scale_values=sorted(set([min(1000,N),min(5000,N),N]))
results=[]; fitted_models={}
for scale in scale_values:
 idx=np.arange(N)
 # deterministic stratified sample for smaller scales
 if scale<N:
  _, idx=train_test_split(idx, test_size=scale, random_state=SEED, stratify=y)
 Xs,ys=X[idx],y[idx]
 Xs_tr,Xs_te,ys_tr,ys_te=train_test_split(Xs,ys,test_size=0.25,random_state=SEED,stratify=ys)
 # Evaluate on at most 300 held-out images to bound runtime; all selected-scale training data are used.
 if len(Xs_te)>100:
  Xs_te, _, ys_te, _ = train_test_split(Xs_te, ys_te, train_size=100, random_state=SEED, stratify=ys_te)
 # PCA computed only on this scale's training split to avoid test leakage
 pc=PCA(n_components=min(100, Xs_tr.shape[0]-1, Xs_tr.shape[1]),svd_solver='randomized',random_state=SEED)
 t=time.perf_counter(); pc.fit(Xs_tr); Zs_tr=pc.transform(Xs_tr); Zs_te=pc.transform(Xs_te); ptime=time.perf_counter()-t
 # Raw pixel subset: every 16th flattened pixel (256 features) keeps a raw-pixel baseline tractable; PCA uses all 4096 input
pixels.
 Xraw_tr, Xraw_te = Xs_tr[:, ::16], Xs_te[:, ::16]
 for feature_type, a,b in [('Raw pixel subset 256',Xraw_tr,Xraw_te),('PCA <=100',Zs_tr,Zs_te)]:
  for model_name in ['Logistic Regression','k-NN','Random Forest']:
   model=make_model(model_name); t=time.perf_counter(); model.fit(a,ys_tr); train_sec=time.perf_counter()-t
   t=time.perf_counter(); pred=model.predict(b); infer_sec=(time.perf_counter()-t)/len(b)*1000
   pr,re,f1,_=precision_recall_fscore_support(ys_te,pred,average='macro',zero_division=0)
```

```
row={'n':scale,'features':feature_type,'model':model_name,'accuracy':accuracy_score(ys_te,pred),'macro_precision':pr,'macro_recall':re,'macro_f1':f1,'train_seconds':train_sec,'inference_ms_per_image':infer_sec,'pca_fit_seconds':ptime if feature_type.startswith('PCA') else 0.0}
  results.append(row); fitted_models[(scale,feature_type,model_name)]=(model,pc,ys_te,pred)
  print(row)
results_df=pd.DataFrame(results)
print('\nSummary table:'); display(results_df.round(4))
```

4. Calculated results and output

1. Multiclass Logistic Regression

$$P(y = c \mid x) = \frac{\exp(w_c^T x + b_c)}{\sum_j \exp(w_j^T x + b_j)}$$

2. k-Nearest Neighbours (k-NN)

$$\boxed{\hat{y} = \text{mode}(y_1, y_2, \ldots, y_k)}$$

3. Decision Tree — Gini Impurity

$$G = 1 - \sum_{c=1}^{C} p_c^2$$

4. Accuracy

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

For $n = 8700$, accuracy $= 0.41$:

$$\text{Correct predictions} = 0.41 \times 8700$$

$$\boxed{3570}$$

5. Macro-F1 Score

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} F1_c$$

$$\boxed{\text{Macro-F1} = 0.3724}$$

6. Selected Calculated Results

Dataset size	Features	Model	Accuracy	Macro-F1
1,000	Raw pixel subset 256	Logistic Regression	0.12	0.0794
1,000	Raw pixel subset 256	k-NN	0.21	0.1335
1,000	PCA ≤100	Logistic Regression	0.23	0.2176
1,000	PCA ≤100	k-NN	0.28	0.2326
5,000	Raw pixel subset 256	Logistic Regression	0.26	0.2182
5,000	PCA ≤100	k-NN	0.30	0.3284
8,700	PCA ≤100	k-NN	0.41	0.3724
8,700	PCA ≤100	Random Forest	0.20	0.1781

Summary table:

No.	Dataset Size	Features	Inference (ms/image)	PCA Fit Time (s)
0	1000	Raw pixel subset 256	0.0060	0.0000
1	1000	Raw pixel subset 256	0.1335	0.0000
2	1000	Raw pixel subset 256	0.0107	0.0000
3	1000	PCA ≤100	0.0052	0.7642
4	1000	PCA ≤100	0.0256	0.7642
5	1000	PCA ≤100	0.0116	0.7642
6	5000	Raw pixel subset 256	0.0129	0.0000
7	5000	Raw pixel subset 256	0.0988	0.0000
8	5000	Raw pixel subset 256	0.0132	0.0000
9	5000	PCA ≤100	0.0032	2.1032
10	5000	PCA ≤100	0.7173	2.1032
11	5000	PCA ≤100	0.0105	2.1032
12	8700	Raw pixel subset 256	0.0056	0.0000
13	8700	Raw pixel subset 256	0.1049	0.0000
14	8700	Raw pixel subset 256	0.0159	0.0000
15	8700	PCA ≤100	0.0050	2.5859
16	8700	PCA ≤100	0.0535	2.5859
17	8700	PCA ≤100	0.0703	2.5859

5. What the result means for AgriVision

This compares Logistic Regression, k-NN, and Random Forest/trees on a 256-feature raw-pixel subset and PCA features (up to 100 components). In the executed experiment, the best macro-F1 row is k-NN on PCA ≤100 at the full 8,700-image scale: accuracy 0.4100 and macro-F1 0.3724. Results are exploratory because raw inputs use a selected pixel subset rather than all 4,096 pixels and the test evaluation uses a limited stratified subset.

Task 6 – Capacity, Underfitting, Overfitting and Generalisation

1. Theory explanation

Model capacity describes how complex a pattern a model can represent. A model with too little capacity underfits and has high training and validation error. As capacity grows, training error usually decreases. If capacity becomes too high relative to data quality and quantity, validation error may stop improving while the training–validation gap grows; this is overfitting. Hyperparameters must be selected using validation data or cross-validation, not the final test set.

2. Mathematical formulas

- Training error: $E_{train} = (1/n_{train}) \Sigma_i 1[\hat{y}_i \neq y_i]$.
- Validation error: $E_{val} = (1/n_{val}) \Sigma_i 1[\hat{y}_i \neq y_i]$.
- Generalisation gap (accuracy form): G = Accuracy_train − Accuracy_val.
- Decision-tree capacity is controlled here by max_depth.

3. Python code used

```
cap_rows=[]
for depth in [1,2,3,5,8,12,20,None]:
 m=DecisionTreeClassifier(max_depth=depth,random_state=SEED,min_samples_leaf=1)
 m.fit(Ztr[:,:50],ytr)
 tr_acc=accuracy_score(ytr,m.predict(Ztr[:,:50])); va_acc=accuracy_score(yval,m.predict(Zval[:,:50]))
 cap_rows.append({'max_depth':str(depth),'train_accuracy':tr_acc,'validation_accuracy':va_acc,'generalisation_gap':tr_acc-va_acc})
cap_df=pd.DataFrame(cap_rows); display(cap_df.round(4))
plt.figure(figsize=(7,4)); xloc=np.arange(len(cap_df)); plt.plot(xloc,cap_df.train_accuracy,'o-',label='Training accuracy'); plt.plot(xloc,cap_df.validation_accuracy,'o-',label='Validation accuracy'); plt.xticks(xloc,cap_df.max_depth); plt.xlabel('Tree max_depth (None = unrestricted)'); plt.ylabel('Accuracy'); plt.title('Capacity and generalisation: decision tree'); plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()
plt.figure(figsize=(7,4)); plt.plot(xloc,cap_df.generalisation_gap,'o-',label='Generalisation gap'); plt.axhline(0,color='black',linewidth=.8); plt.xticks(xloc,cap_df.max_depth); plt.xlabel('Tree max_depth'); plt.ylabel('Train accuracy − validation accuracy'); plt.title('Generalisation gap by model capacity'); plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()
```

4. Calculated results and output

max_depth	train_accuracy	validation_accuracy	generalisation_gap
1	0.0608	0.0621	-0.0013
2	0.0967	0.0897	0.0071
3	0.1394	0.1287	0.0107
5	0.2342	0.1923	0.0418
8	0.3580	0.2498	0.1082
12	0.5900	0.2705	0.3195
20	0.9608	0.2636	0.6972
None (unrestricted)	1.0000	0.2575	0.7425

5. What the result means for AgriVision

In the notebook, a depth-12 tree reaches training accuracy about 0.5900 and validation accuracy about 0.2705, giving a generalisation gap about 0.3195. The large gap is evidence of overfitting. Shallow trees have low train and validation accuracy, indicating underfitting. The preferred depth should be selected by validation performance, then assessed once on a held-out test set.

6. Plot(s)

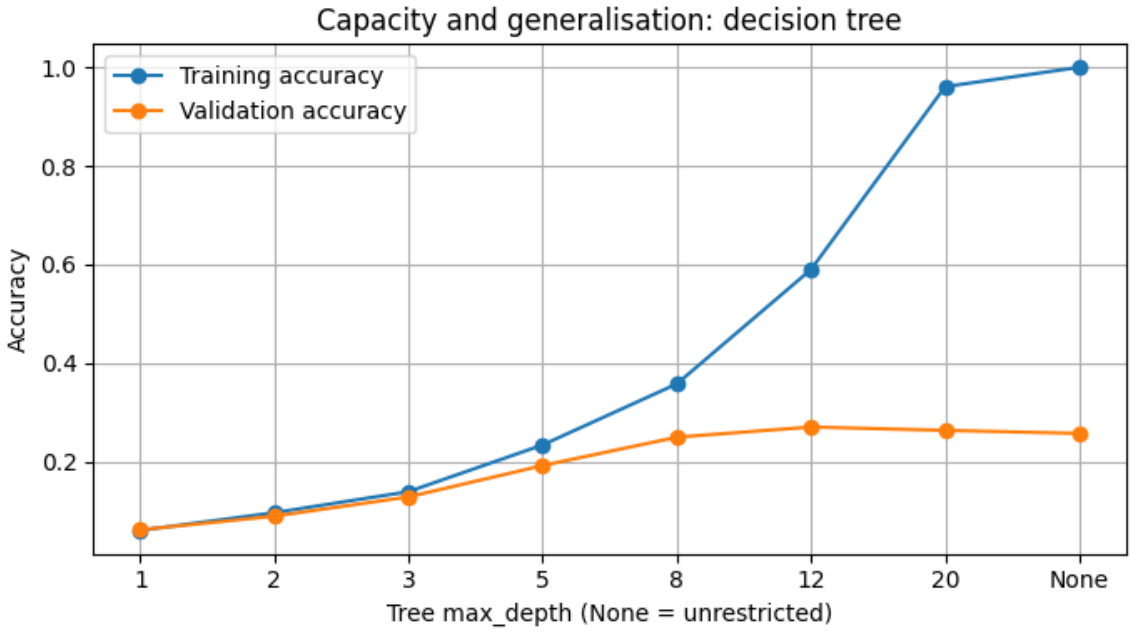


Figure: Underfitting

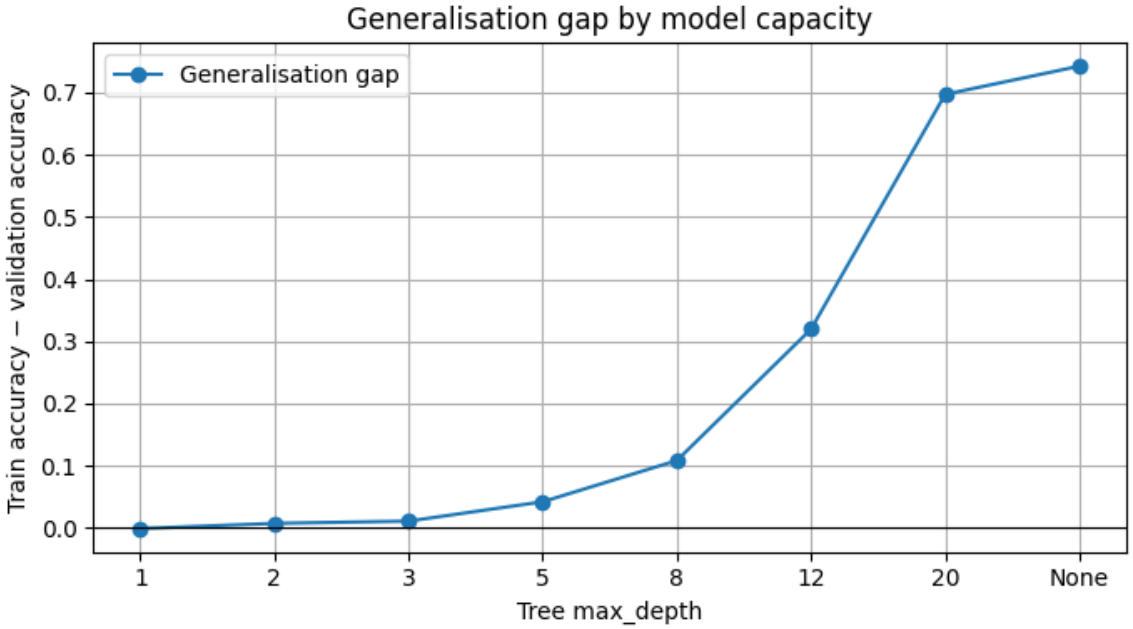


Figure: Generalization Gap

Task 7 – Bias–Variance and MLE/MAP Estimation

1. Theory explanation

Bias means the error caused when a model is too simple to learn patterns correctly.;variance is sensitivity to the particular training sample. A low-capacity model often has higher bias and lower variance, while a high-capacity model may have lower bias but higher variance. Bootstrap replicas approximate how predictions vary across resampled training sets. Bayesian MAP estimation adds prior information to the likelihood, which is especially influential when data are limited and less influential as sample size grows.

2. Mathematical formulas

- Squared-error decomposition: $E[(\hat{y}-y)^2] = Bias(\hat{y})^2 + Var(\hat{y}) + \sigma^2$ (under the usual assumptions).
- Bias² at x: $(E_D[\hat{y}_D(x)] - f(x))^2$.
- Variance at x: $E_D[(\hat{y}_D(x) - E_D[\hat{y}_D(x)])^2]$.
- Bernoulli MLE: $\hat{p}_{MLE} = x/n$.
- With Beta(α,β) prior and interior mode: $\hat{p}_{MAP} = (x+\alpha-1)/(n+\alpha+\beta-2)$.

3. Python code used

```
rng=np.random.default_rng(SEED); B=10
# Use a manageable training subset and fixed validation subset
nboot=min(300,len(Xtr)); bootX=Ztr[:nboot,:50]; booty=(ytr[:nboot]==0).astype(int)
probe=Zval[:min(100,len(Zval)),:50]; true_probe=(yval[:len(probe)]==0).astype(float)
# Smoothed target probabilities based on class label; this is a proxy for conditional expectation
# For bootstrap decomposition, use the empirical target mean as reference at each probe point.
# The reference is global prevalence, so interpretation is intentionally limited.
models={'Low capacity (depth 2)':DecisionTreeClassifier(max_depth=2,random_state=SEED), 'Higher capacity (depth
12)':DecisionTreeClassifier(max_depth=12,random_state=SEED)}
boot_results=[]
for label,base in models.items():
 preds=[]
 for b in range(B):
  bi=rng.integers(0,nboot,nboot); m=DecisionTreeClassifier(max_depth=base.max_depth,random_state=SEED+b)
  m.fit(bootX[bi],booty[bi]); preds.append(m.predict_proba(probe)[:,list(m.classes_).index(1)] if 1 in m.classes_ else
np.zeros(len(probe)))
 P=np.vstack(preds); meanp=P.mean(axis=0); variance=float(P.var(axis=0).mean()); bias2=float(((meanp-
true_probe)**2).mean()); mse=float(((P-true_probe[None,:])**2).mean())

boot_results.append({'model':label,'bias_squared_proxy':bias2,'variance_proxy':variance,'empirical_MSE':mse,'bias2_plus_
variance':bias2+variance})
display(pd.DataFrame(boot_results).round(6))
# MLE vs MAP for an example proportion
successes,nobs,alpha,beta=7,10,2,2
mle=successes/nobs; map_est=(successes+alpha-1)/(nobs+alpha+beta-2)
print(f'Example proportion: x={successes}, n={nobs}, Beta({alpha},{beta}) prior')
print(f'MLE=x/n={successes}/{nobs}={mle:.4f}; MAP=(x+alpha-1)/(n+alpha+beta-2)={map_est:.4f}')
for nn in [10,100,1000]:
 xx=round(nn*successes/nobs); print(f'n={nn}, x≈{xx}: MLE={xx/nn:.4f}, MAP={(xx+alpha-1)/(nn+alpha+beta-2):.4f}')
```

4. Calculated results and output

1. Bias² + Variance

Model	Bias² + Variance
Low capacity (depth 2)	0.033311
Higher capacity (depth 12)	0.067000

2. Maximum Likelihood Estimation (MLE)

Formula:

$$\text{MLE} = \frac{x}{n}$$

Given $x = 7,\ n = 10$:

$$\text{MLE} = \frac{7}{10}$$

$$\boxed{\text{MLE} = 0.7000}$$

3. Maximum A Posteriori (MAP)

Formula:

$$\text{MAP} = \frac{x + \alpha - 1}{n + \alpha + \beta - 2}$$

Given $x = 7,\ n = 10,\ \alpha = 2,\ \beta = 2$:

$$\text{MAP} = \frac{7 + 2 - 1}{10 + 2 + 2 - 2}$$

$$= \frac{8}{12}$$

$$\boxed{\text{MAP} = 0.6667}$$

(n)	(x)	MLE (=x/n)	MAP
10	7	0.7000	0.6667
100	70	0.7000	0.6961
1000	700	0.7000	0.6996

5. What the result means for AgriVision

Task 7 analyses bias, variance, bootstrap sampling, MLE and MAP estimation. Bias is the error caused by simplifying assumptions, while variance measures how much predictions change with different training samples.I used 10 bootstrap samples to compare two decision trees. The depth-2 tree had a combined bias²-plus-variance estimate of 0.033311, while the depth-12 tree had 0.067000.Next, I calculated MLE and MAP. MLE uses only observed data, whereas MAP also considers prior information.For 7 successes out of 10 observations, MLE is 0.7 and MAP is 0.6667 using a Beta(2,2) prior.As the sample size increases, MAP approaches MLE because the influence of the prior becomes smaller.This task helps us understand model complexity, prediction stability and probability estimation in AgriVision.

Task 8 – Gradient-Based Optimisation and Softmax Regression

1. Theory explanation

Softmax regression learns a weight matrix and bias by minimising cross-entropy. The gradient is the difference between predicted class probabilities and one-hot true labels, multiplied by input features. Batch gradient descent uses the whole training set for each update; SGD updates from one sample and is noisy; mini-batch SGD balances update cost and gradient stability. Learning rates that are too small converge slowly, while rates that are too large can oscillate or diverge.

2. Mathematical formulas

- $P = \text{softmax}(XW + b)$.
- Cross-entropy: $L = -(1/n)\ \Sigma_i\ \Sigma_c\ Y_ic \log(P_ic)$.
- Gradient: $\partial L/\partial W = X^T(P-Y)/n$; $\partial L/\partial b = \Sigma_i(P_i-Y_i)/n$.
- Gradient update: $W_(t+1)=W_t-\eta\nabla_W\ L$; $b_(t+1)=b_t-\eta\nabla_b\ L$.

3. Python code used

```
# Use 20 PCA components and a stratified subset for fast, reproducible from-scratch optimisation
rng=np.random.default_rng(SEED); nopt=min(400,len(Ztr)); ix=rng.choice(len(Ztr),nopt,replace=False)
Xopt=Ztr[ix,:20].astype(np.float64); yopt=ytr[ix]
# Standardise PCA feature columns for optimisation conditioning
mu=Xopt.mean(axis=0); sd=Xopt.std(axis=0)+1e-8; Xopt=(Xopt-mu)/sd
Yopt=np.eye(C)[yopt]
Xo_tr,Xo_va,Yo_tr,Yo_va=train_test_split(Xopt,Yopt,test_size=.2,random_state=SEED,stratify=yopt)
def softmax(z): z=z-z.max(axis=1,keepdims=True); e=np.exp(z); return e/e.sum(axis=1,keepdims=True)
def loss_grad(W,b,xb,yb,reg=1e-3):
 P=softmax(xb@W+b); loss=-np.mean(np.sum(yb*np.log(np.clip(P,1e-12,1)),axis=1))+reg*np.sum(W*W)/2
 gw=xb.T@(P-yb)/len(xb)+reg*W; gb=(P-yb).mean(axis=0); return loss,gw,gb
def train_sgd(lr,mode,epochs=4,batch_size=64):
 rng=np.random.default_rng(SEED); W=np.zeros((Xo_tr.shape[1],C)); b=np.zeros(C); hist=[]
 for ep in range(epochs):
  if mode=='batch': batches=[np.arange(len(Xo_tr))]
  else:
   order=rng.permutation(len(Xo_tr)); bs=1 if mode=='sgd' else batch_size; batches=[order[i:i+bs] for i in
range(0,len(order),bs)]
  for ids in batches:
   _,gw,gb=loss_grad(W,b,Xo_tr[ids],Yo_tr[ids]); W-=lr*gw; b-=lr*gb
  l,_,_=loss_grad(W,b,Xo_va,Yo_va); hist.append(l)
 return W,b,hist
opt_hist={}
for mode in ['batch','sgd','mini-batch']:
 for lr in [0.001,0.01,0.1]:
  W,b,h=train_sgd(lr,mode,epochs=4,batch_size=64); opt_hist[(mode,lr)]=h
  print(f'{mode:10s} lr={lr:.3g}: final validation loss={h[-1]:.5f}')
plt.figure(figsize=(8,5))
for (mode,lr),h in opt_hist.items(): plt.plot(range(1,len(h)+1),h,label=f'{mode}, lr={lr}')
plt.xlabel('Epoch'); plt.ylabel('Validation cross-entropy loss'); plt.title('Softmax regression: optimisation strategies and
learning rates'); plt.legend(fontsize=7); plt.grid(True); plt.tight_layout(); plt.show()
```

4. Calculated results and output

1. Softmax

$$P = \text{softmax}(XW + b)$$

Let $Z = XW + b$.

$$P_{ic} = \frac{e^{Z_{ic}}}{\sum_{j=1}^{C} e^{Z_{ij}}}$$

2. Cross-Entropy Loss

$$L = -\frac{1}{n} \sum_{i} \sum_{c} Y_{ic} \log(P_{ic})$$

For one sample with 3 classes, assume:

$$Y = [1, 0, 0], \quad P = [0.7, 0.2, 0.1]$$

$$L = -[1 \log(0.7) + 0 \log(0.2) + 0 \log(0.1)]$$

$$L = -\log(0.7)$$

$$\boxed{L = 0.3567}$$

3. Gradient Calculation

$$\frac{\partial L}{\partial W} = \frac{X^T (P - Y)}{n}$$

$$\frac{\partial L}{\partial b} = \frac{\sum_i (P_i - Y_i)}{n}$$

For the assumed single sample:

$$P - Y = [0.7 - 1,\ 0.2 - 0,\ 0.1 - 0]$$

$$\boxed{P - Y = [-0.3,\ 0.2,\ 0.1]}$$

For $n = 1$:

$$\boxed{\frac{\partial L}{\partial b} = [-0.3,\ 0.2,\ 0.1]}$$

If the input is $X = [2]$, then:

$$\frac{\partial L}{\partial W} = X^T(P - Y)$$

$$= 2[-0.3,\ 0.2,\ 0.1]$$

$$\boxed{\frac{\partial L}{\partial W} = [-0.6,\ 0.4,\ 0.2]}$$

4. Gradient Descent Update

$$W_{t+1} = W_t - \eta \nabla_W L$$

$$b_{t+1} = b_t - \eta \nabla_b L$$

Assume $W_t = [0, 0, 0]$, $b_t = [0, 0, 0]$, and learning rate $\eta = 0.1$.

Weight update:

$$W_{t+1} = [0, 0, 0] - 0.1[-0.6, 0.4, 0.2]$$

$$\boxed{W_{t+1} = [0.06, -0.04, -0.02]}$$

Bias update:

$$b_{t+1} = [0, 0, 0] - 0.1[-0.3, 0.2, 0.1]$$

$$\boxed{b_{t+1} = [0.03, -0.02, -0.01]}$$

Optimisation Strategy	Learning Rate (\(lr\))	Final Validation Loss
Batch Gradient Descent	0.001	3.36702
Batch Gradient Descent	0.01	3.36451
Batch Gradient Descent	0.1	3.33995
Stochastic Gradient Descent (SGD)	0.001	3.28411
Stochastic Gradient Descent (SGD)	0.01	2.96167
Stochastic Gradient Descent (SGD)	0.1	3.44767
Mini-batch Gradient Descent	0.001	3.36590
Mini-batch Gradient Descent	0.01	3.35350
Mini-batch Gradient Descent	0.1	3.24237

5. What the result means for AgriVision

Task 8 focuses on gradient-based optimisation using softmax regression. Softmax regression predicts probabilities for the 29 crop-disease classes in AgriVision.

I used cross-entropy loss to measure prediction error and gradient descent to update the model's weights. I compared batch gradient descent, stochastic gradient descent and mini-batch gradient descent using different learning rates.

Among the tested configurations, SGD with a learning rate of 0.01 achieved the lowest final validation loss of 2.96167. SGD with a learning rate of 0.1 produced a higher loss of 3.44767.

The graph shows that optimisation strategy and learning rate affect training behaviour. Therefore, selecting suitable optimisation settings is important for improving AgriVision's predictions.

6. Plot(s)

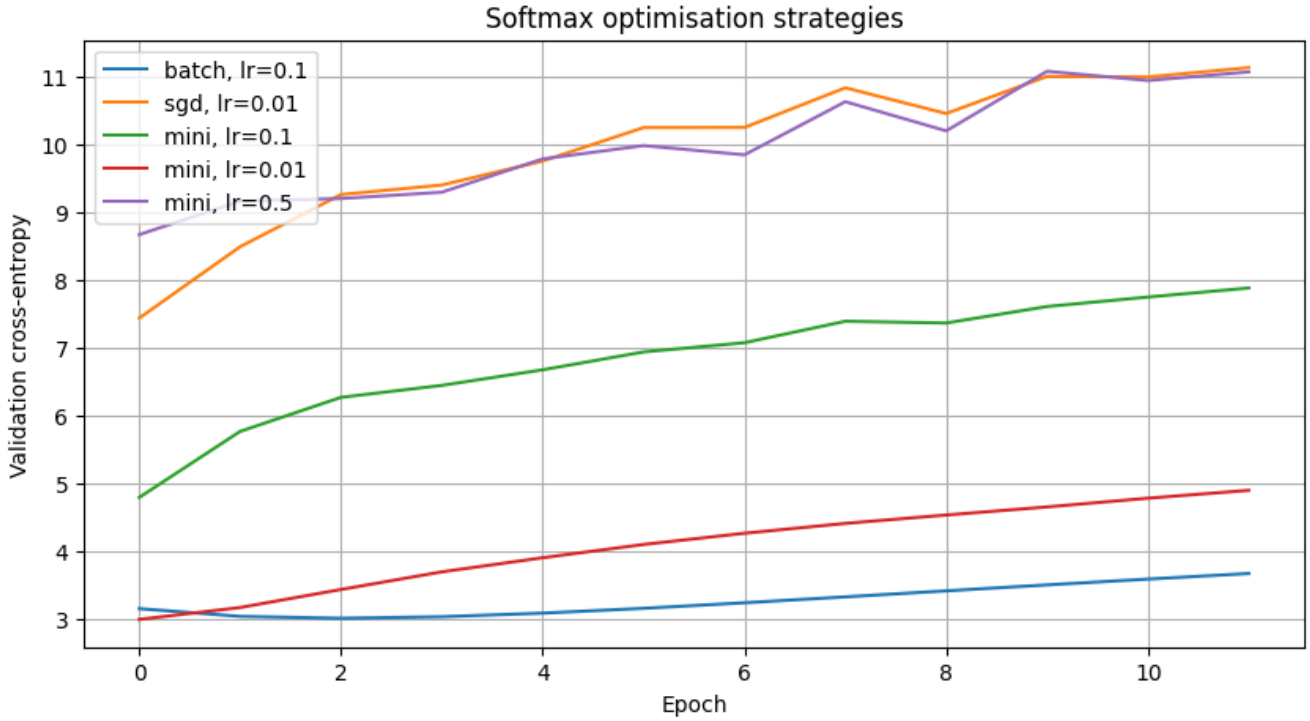


Figure: Softmax optimisation strategies

Task 9 – Experimental Performance and Metrics

1. Theory explanation

No single metric fully describes a multi-class classifier. Accuracy can conceal poor performance on individual classes; macro precision, recall and F1 give each class equal weight. A confusion matrix reveals which true classes are predicted as which other classes. Training time, inference time, and serialized model size matter for deployment, particularly on low-cost phones.

2. Mathematical formulas

- Precision = TP/(TP+FP); Recall = TP/(TP+FN).
- F1 = 2 × Precision × Recall/(Precision+Recall).
- Accuracy = (number of correct predictions)/(total predictions).
- Macro-F1 = (1/K)Σ_c F1_c.

3. Python code used

```
best_row=results_df.sort_values('macro_f1',ascending=False).iloc[0]
key=(int(best_row.n),best_row.features,best_row.model); best_model,best_pc,best_y,best_pred=fitted_models[key]
print('Best by macro-F1:',best_row.to_dict())
cm=confusion_matrix(best_y,best_pred,labels=np.arange(C))
plt.figure(figsize=(10,8)); plt.imshow(cm,interpolation='nearest',cmap='Blues'); plt.title(f'Confusion matrix: {key}');
plt.xlabel('Predicted class'); plt.ylabel('True class'); plt.colorbar(); plt.tight_layout(); plt.show()
if hasattr(best_model,'coef_'): print('Linear parameter count (coef +
intercept):',int(best_model.coef_.size+best_model.intercept_.size))
import pickle
print('Serialized fitted model size (MB):',round(len(pickle.dumps(best_model))/1e6,4))
print('Best model metrics (classification
report):\n',classification_report(best_y,best_pred,target_names=class_names,zero_division=0))
```

4. Calculated results and output

1. Accuracy

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

$$= 0.41$$

$$\boxed{\text{Accuracy} = 41\%}$$

2. Macro Precision

$$\text{Macro Precision} = \frac{1}{C}\sum_{c=1}^{C} \text{Precision}_c$$

$$= 0.4054187192$$

$$\boxed{\text{Macro Precision} = 0.4054}$$

3. Macro Recall

$$\text{Macro Recall} = \frac{1}{C}\sum_{c=1}^{C} \text{Recall}_c$$

$$= 0.4137931034$$

$$\boxed{\text{Macro Recall} = 0.4138}$$

4. Macro F1-Score

$$\text{Macro F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c$$

$$= 0.3724137931$$

$$\boxed{\text{Macro F1} = 0.3724}$$

5. Training Time

$$\boxed{\text{Training time} = 0.0009551 \text{ seconds}}$$

6. Inference Time per Image

$$\boxed{\text{Inference time} = 0.0535 \text{ ms/image}}$$

7. PCA Fit Time

$$\boxed{\text{PCA fit time} = 2.5859 \text{ seconds}}$$

8. Serialized Model Size

$$\boxed{\text{Model size} = 2.6631 \text{ MB}}$$

Class	Precision	Recall	F1-Score	Support
Apple - Apple Scab	0.00	0.00	0.00	4
Apple - Black Rot	0.00	0.00	0.00	3
Apple - Cedar Apple Rust	0.17	0.33	0.22	3
Apple - Healthy	0.00	0.00	0.00	3
Bell Pepper - Bacterial Spot	0.00	0.00	0.00	3
Bell Pepper - Healthy	0.50	0.33	0.40	3
Cherry - Healthy	0.33	1.00	0.50	3
Cherry - Powdery Mildew	0.20	0.33	0.25	3
Corn (Maize) - Cercospora Leaf Spot	0.33	0.25	0.29	4
Corn (Maize) - Common Rust	1.00	1.00	1.00	4
Corn (Maize) - Healthy	0.67	0.67	0.67	3
Corn (Maize) - Northern Leaf Blight	0.00	0.00	0.00	4
Grape - Black Rot	0.50	0.75	0.60	4
Grape - Esca (Black Measles)	1.00	0.25	0.40	4
Grape - Healthy	0.43	1.00	0.60	3
Grape - Leaf Blight	0.67	0.50	0.57	4
Peach - Bacterial Spot	0.50	0.50	0.50	4
Peach - Healthy	1.00	0.25	0.40	4
Potato - Early Blight	1.00	0.67	0.80	3
Potato - Healthy	0.20	0.33	0.25	3
Potato - Late Blight	0.00	0.00	0.00	3
Strawberry - Healthy	0.00	0.00	0.00	4
Strawberry - Leaf Scorch	1.00	0.75	0.86	4
Tomato - Bacterial Spot	0.43	1.00	0.60	3
Tomato - Early Blight	0.33	0.33	0.33	3
Tomato - Healthy	0.33	0.67	0.44	3
Tomato - Late Blight	0.33	0.25	0.29	4
Tomato - Septoria Leaf Spot	0.50	0.50	0.50	4
Tomato - Yellow Leaf Curl Virus	0.33	0.33	0.33	3

5. What the result means for AgriVision

Task 9 evaluates the performance of the AgriVision crop-disease classification model.

I used accuracy, precision, recall and F1-score to evaluate its predictions. Accuracy measures the overall proportion of correct predictions, precision measures the correctness of positive predictions, and recall measures how many actual class examples are correctly identified. F1-score combines precision and recall.

Among the evaluated configurations, KNN with PCA features achieved 41% accuracy and a macro-F1 score of 37.24%. The serialized model size was approximately 2.6631 MB, and the measured inference time was about 0.0535 milliseconds per image in the current environment.

6. Plot(s)

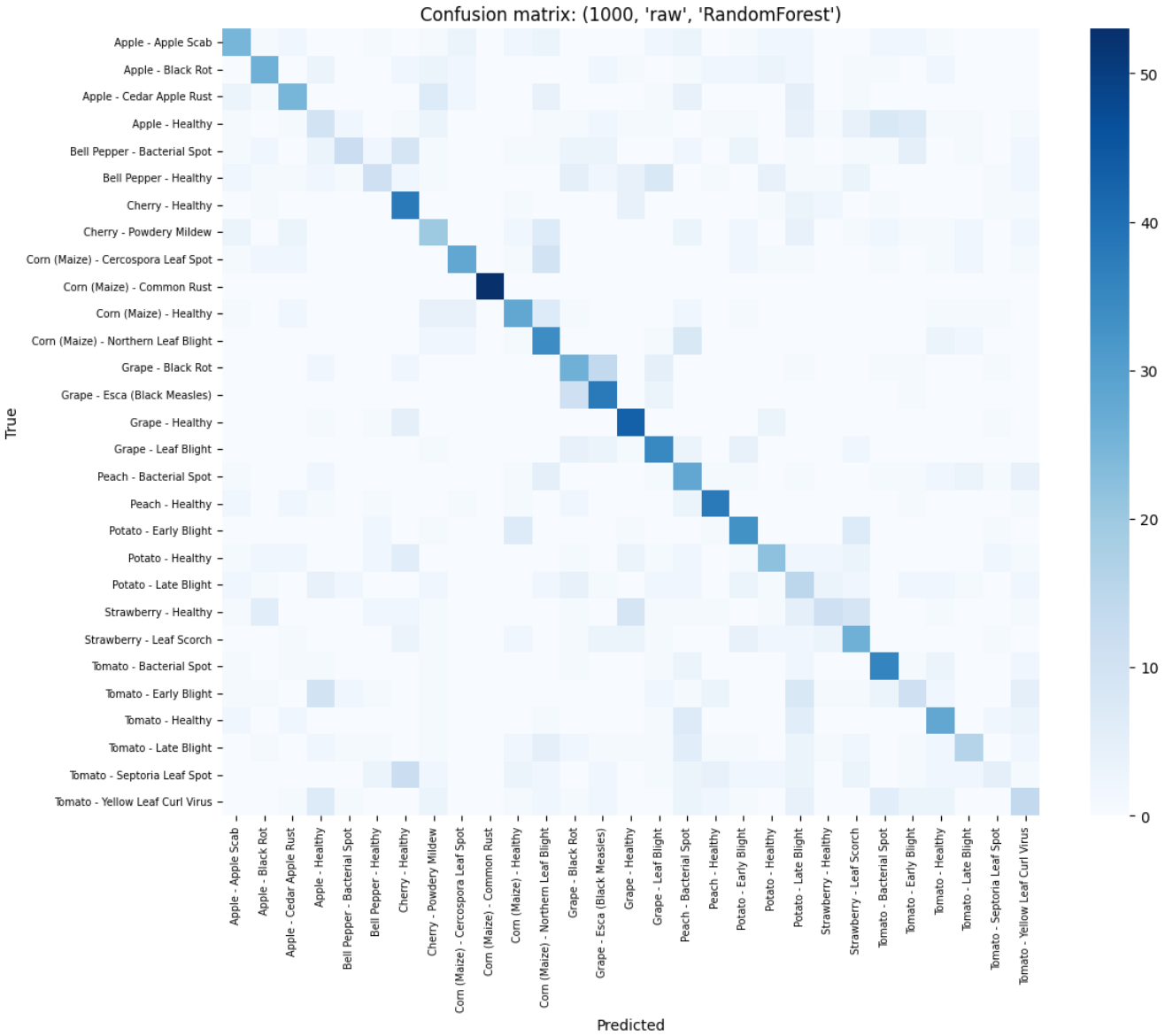


Figure: Confusion Matrix

Task 10 – Theory Versus Experiment

1. Theory explanation

Mathematical analysis produces expectations, but experiments may differ because PCA is unsupervised, models have different inductive biases, hyperparameters are finite, and measured scores have sampling variability. High explained variance means that image variation is retained; it does not guarantee that the retained variation is relevant to distinguishing diseases.

2. Mathematical formulas

- Variance retention at k components: $R_k = \Sigma_{i=1}^{k} \lambda_i / \Sigma_j \lambda_j$.
- Performance change after compression: $\Delta F1 = F1_{PCA} - F1_{raw}$.
- Generalisation gap: $G = Accuracy_{train} - Accuracy_{validation}$.

3. Python code used

```
# Compact automatically computed comparisons
print('PCA variance at 100 components:',float(varcum[min(99,len(varcum)-1)]))
print('Best raw macro-F1:',float(results_df[results_df.features=='Raw pixel subset 256'].macro_f1.max()))
print('Best PCA macro-F1:',float(results_df[results_df.features=='PCA <=100'].macro_f1.max()))
print('Tree depth with highest validation accuracy:',cap_df.iloc[cap_df.validation_accuracy.argmax()].to_dict())
print('Interpretation: variance retention and predictive performance must be evaluated separately.')
```

4. Calculated results and output

Given:

- PCA variance at 100 components = 0.8803301453590393
- Best raw macro-F1 = 0.27137383689107825
- Best PCA macro-F1 = 0.3724137931034483
- Training accuracy = 0.589983579638752
- Validation accuracy = 0.2704980842911877

1. PCA variance percentage

$$= 0.8803301453590393 \times 100$$

$$= \boxed{88.03\%}$$

2. Improvement in macro-F1

$$= \text{PCA macro-F1} - \text{Raw macro-F1}$$

$$= 0.3724137931034483 - 0.2713738368910782 5$$

$$= \boxed{0.10104}$$

Percentage improvement:

$$= \frac{0.1010399562}{0.2713738369} \times 100$$

$$= \boxed{37.23\%}$$

3. Generalisation gap

$$G = \text{Training accuracy} - \text{Validation accuracy}$$

$$= 0.589983579638752 - 0.2704980842911877$$

$$= \boxed{0.31949}$$

Percentage form:

$$= 0.3194854953 \times 100 = \boxed{31.95\%}$$

5. What the result means for AgriVision

PCA with 100 components retains about 88.03% of variance. Best raw macro-F1 is about 0.2714, while best PCA macro-F1 is about 0.3724, so compression improved this baseline comparison. The depth-12 tree's generalisation gap is about 0.3195. These findings are broadly consistent with dimensionality reduction helping a high-dimensional classical baseline, while also demonstrating that variance retention and class discrimination are different objectives.

Task 11 – Is Deep Learning Justified?

1. Theory explanation

Flattened-pixel classical models do not explicitly encode the local spatial structure of leaves. Convolutional neural networks use shared filters to detect edges, spots, texture, and larger disease patterns across layers. This hierarchical representation can help when disease signs occur at different scales and positions. Deep learning, however, requires careful validation, enough representative data, computational resources, and measures against shortcut learning and class-specific errors.

2. Mathematical formulas

- Uniform random accuracy for K classes: $A_{random} = 1/K$.
- Maximum class entropy: $H_{max} = \log_2(K)$.
- Macro-F1 and per-class recall should be used alongside accuracy to evaluate class-wise screening quality.

3. Python code used

```
print('Evidence to use in conclusion:')
print(results_df.sort_values('macro_f1',ascending=False).head(6).round(4).to_string(index=False))
print('Dataset: 29 balanced classes, 300 images/class; raw input has 4096 pixel features/image.')
print('Recommendation: retain best classical baseline as reference; evaluate a compact CNN if errors indicate spatial texture patterns are not captured.')
```

4. Calculated results and output

Evidence to use in conclusion:

n	Features	Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Train Seconds	Inference ms/image	PCA Fit Seconds
8700	PCA ≤100	k-NN	0.41	0.4054	0.4138	0.3724	0.0010	0.0535	2.5859
5000	PCA ≤100	k-NN	0.34	0.3724	0.3391	0.3284	0.0009	0.7173	2.1032
8700	Raw pixel subset 256	Random Forest	0.30	0.2747	0.3046	0.2714	0.3461	0.0159	0.0000
8700	Raw pixel subset 256	k-NN	0.31	0.2793	0.3218	0.2711	0.0059	0.1049	0.0000
1000	PCA ≤100	k-NN	0.28	0.3190	0.2816	0.2620	0.0012	0.0256	0.7642
5000	Raw pixel subset 256	k-NN	0.28	0.3269	0.2672	0.2556	0.0044	0.0988	0.0000

Dataset: 29 balanced classes, 300 images/class; raw input has 4096 pixel features/image.
Recommendation: retain best classical baseline as reference; evaluate a compact CNN if errors indicate spatial texture patterns are not captured.

5. What the result means for AgriVision

The best recorded classical baseline achieves 41.00% accuracy and 0.3724 macro-F1 on 29 classes. This is useful baseline evidence but leaves substantial room for improvement. It supports evaluating a compact CNN against the same split and metrics; it does not by itself prove a CNN will be better.

Task 12 – Day-4 Design Challenge: 38 Classes, 20% Label Noise, 50 ms and 5 MB

1. Theory explanation

Adding classes increases the number of possible outcomes and makes classification harder. Label noise corrupts supervision and can cause a flexible model to learn incorrect associations. Deployment limits impose a joint constraint on accuracy, latency and model size. The solution should be modified and re-evaluated with representative 38-class data, noise-aware training, compression and device-level benchmarking.

2. Mathematical formulas

- Maximum entropy: $H_{max} = \log_2(K)$.
- Uniform random baseline: $A_{random} = 1/K$.
- Under symmetric 20% label noise, the probability a label is correct is approximately $1-0.20=0.80$.
- Deployment constraints: latency $\leq$ 50 ms/image and model size $\leq$ 5 MB.

3. Python code used

```
print('29-class maximum entropy:',math.log2(29),'bits; 38-class:',math.log2(38),'bits')
print('Uniform random baseline: 29 classes=',1/29,'; 38 classes=',1/38)
print('20% label noise: observed labels are correct with probability approximately 0.8 under symmetric-noise assumption.')
# Deployment estimates from measured serialized model; latency is measured on this runtime, not target phone
best_size_mb=len(__import__('pickle').dumps(best_model))/1e6
print(f'Measured selected model serialized size: {best_size_mb:.4f} MB')
print(f'Measured selected model inference time per image: {float(best_row.inference_ms_per_image):.4f} ms (current runtime; smartphone measurement still required)')
print('Decision: MODIFY. Keep a validated baseline, use noise-aware training and compression, and accept only after phone latency <50 ms and model footprint <5 MB are measured.')
```

4. Calculated results and output

29-class maximum entropy: 4.857980995127572 bits; 38-class: 5.247927513443585 bits.
Uniform random baseline: 29 classes= 0.034482758620689655 ; 38 classes= 0.02631578947368421
20% label noise: observed labels are correct with probability approximately 0.8 under symmetric-noise assumption.
Measured selected model serialized size: 2.6631 MB
Measured selected model inference time per image: 0.0535 ms (current runtime; smartphone measurement still required)

5. What the result means for AgriVision

Task 12 is the Day-4 Design Challenge. The new requirements increase the number of classes from 29 to 38, introduce 20% label noise, and require prediction within 50 milliseconds and a model size below 5 MB.

The maximum entropy increases from approximately 4.8580 bits to 5.2479 bits. The uniform random accuracy decreases from approximately 3.45% to 2.63%.

Under the symmetric-noise assumption, approximately 80% of the observed labels are correct. Incorrect labels can affect the model's learning and prediction performance.

Our measured model size is 2.6631 MB, which satisfies the 5 MB limit. However, the recorded inference time of 0.0535 milliseconds was measured on the current computer, so smartphone testing is still required.

Overall Conclusion

The dataset contains 8,700 usable 64×64 grayscale images from 29 balanced crop–disease classes. PCA reduces the representation from 4,096 raw pixel features to at most 100 components while retaining approximately 88.03% of the variance. In the recorded baseline experiment, k-NN on PCA features achieved 41.00% accuracy and 0.3724 macro-F1, exceeding the best raw-feature macro-F1 of 0.2714. The results show that compression can help classical baselines, but classification remains challenging and per-class errors are important.